

Optimización de latencia en un clúster de microservicios mediante interpolación de Lagrange

Estimación de la latencia media para una asignación de 6 GB de RAM

Estudiante: **Fernando Istaña Flores** Escuela: **Ingeniería de Sistemas**

1. Contexto y datos

En una arquitectura de microservicios desplegada sobre la nube, el consumo de memoria RAM asignado al contenedor de procesamiento en tiempo real (x , en gigabytes) impacta directamente sobre la latencia media de respuesta de las peticiones HTTP (y , en milisegundos). El comportamiento no es lineal: la memoria caché reduce la latencia hasta cierto punto, pero a partir de un límite los tiempos de recolección de basura vuelven a incrementarla. Durante las pruebas de carga el equipo de DevOps registró cuatro puntos métricos discretos.

PUNTO	MEMORIA ASIGNADA x (GB)	LATENCIA MEDIA $y = f(x)$ (ms)
x_0	2	150
x_1	4	85
x_2	8	50
x_3	12	70

Con cuatro nodos corresponde un polinomio interpolador de grado $n = 3$. El objetivo es estimar la latencia cuando el contenedor se escala a $x = 6$ GB, antes de desplegar esa configuración en producción.

2. Parte A — Construcción de los polinomios base de Lagrange

El polinomio fundamental de Lagrange asociado al nodo k se define como el producto

$$L_k(x) = \prod_{i \neq k} (x - x_i) / (x_k - x_i), \quad k \in \{0, 1, 2, 3\}$$

Cada base vale 1 en su propio nodo y 0 en los demás, que es la propiedad que hace que el polinomio pase exactamente por los cuatro puntos medidos.

Denominadores

$$\begin{aligned} d_0 &= (2-4)(2-8)(2-12) = (-2)(-6)(-10) = -120 \\ d_1 &= (4-2)(4-8)(4-12) = (2)(-4)(-8) = 64 \\ d_2 &= (8-2)(8-4)(8-12) = (6)(4)(-4) = -96 \\ d_3 &= (12-2)(12-4)(12-8) = (10)(8)(4) = 320 \end{aligned}$$

Expresión de cada base

$$L_0(x) = (x-4)(x-8)(x-12) / (-120) = -x^3/120 + x^2/5 - 22x/15 + 16/5$$

$$L_1(x) = (x-2)(x-8)(x-12) / 64 = x^3/64 - 11x^2/32 + 17x/8 - 3$$

$$L_2(x) = (x-2)(x-4)(x-12) / (-96) = -x^3/96 + 3x^2/16 - 5x/6 + 1$$

$$L_3(x) = (x-2)(x-4)(x-8) / 320 = x^3/320 - 7x^2/160 + 7x/40 - 1/5$$

3. Parte A — Formulación del polinomio interpolador $P_3(x)$

$$P_3(x) = \sum_{k=0}^3 y_k \cdot L_k(x) = 150 \cdot L_0(x) + 85 \cdot L_1(x) + 50 \cdot L_2(x) + 70 \cdot L_3(x)$$

Paso 1 — Expandir cada numerador

$$(x-4)(x-8)(x-12) = (x^2-12x+32)(x-12) = x^3 - 24x^2 + 176x - 384$$

$$(x-2)(x-8)(x-12) = (x^2-10x+16)(x-12) = x^3 - 22x^2 + 136x - 192$$

$$(x-2)(x-4)(x-12) = (x^2-6x+8)(x-12) = x^3 - 18x^2 + 80x - 96$$

$$(x-2)(x-4)(x-8) = (x^2-6x+8)(x-8) = x^3 - 14x^2 + 56x - 64$$

Paso 2 — Multiplicar cada numerador por el factor y_k / d_k

K	Y_K / D_K	$Y_K \cdot L_K(x)$
0	$150 / (-120) = -5/4$	$-(5/4)x^3 + 30x^2 - 220x + 480$
1	$85 / 64$	$(85/64)x^3 - (935/32)x^2 + (1445/8)x - 255$
2	$50 / (-96) = -25/48$	$-(25/48)x^3 + (75/8)x^2 - (125/3)x + 50$
3	$70 / 320 = 7/32$	$(7/32)x^3 - (49/16)x^2 + (49/4)x - 14$

Paso 3 — Sumar los coeficientes de igual grado

$$x^3: -5/4 + 85/64 - 25/48 + 7/32 = (-240 + 255 - 100 + 42) / 192 = \mathbf{-43/192}$$

$$x^2: 30 - 935/32 + 75/8 - 49/16 = (960 - 935 + 300 - 98) / 32 = \mathbf{227/32}$$

$$x^1: -220 + 1445/8 - 125/3 + 49/4 = (-5280 + 4335 - 1000 + 294) / 24 = \mathbf{-1651/24}$$

$$x^0: 480 - 255 + 50 - 14 = \mathbf{261}$$

Los denominadores comunes usados son 192 para el término cúbico, 32 para el cuadrático y 24 para el lineal.

$$\mathbf{P_3(x) = - (43/192) \cdot x^3 + (227/32) \cdot x^2 - (1651/24) \cdot x + 261}$$

$$\text{En decimales: } P_3(x) \approx -0.223958 \cdot x^3 + 7.093750 \cdot x^2 - 68.791667 \cdot x + 261.$$

Comprobación: al evaluar en los cuatro nodos se recuperan exactamente los valores medidos (150, 85, 50 y 70), como se verifica en la salida del programa.

4. Parte A — Estimación de la latencia en $x = 6$ GB

Conviene evaluar las bases antes de sumar, porque el reemplazo numérico es más corto que expandir el polinomio.

$$L_0(6) = (6-4)(6-8)(6-12) / (-120) = (2)(-2)(-6) / (-120) = 24 / (-120) = -0.20$$

$$L_1(6) = (6-2)(6-8)(6-12) / 64 = (4)(-2)(-6) / 64 = 48 / 64 = 0.75$$

$$L_2(6) = (6-2)(6-4)(6-12) / (-96) = (4)(2)(-6) / (-96) = -48 / (-96) = 0.50$$

$$L_3(6) = (6-2)(6-4)(6-8) / 320 = (4)(2)(-2) / 320 = -16 / 320 = -0.05$$

Control: $L_0(6) + L_1(6) + L_2(6) + L_3(6) = -0.20 + 0.75 + 0.50 - 0.05 = 1$, como exige la propiedad de partición de la unidad de las bases de Lagrange.

K	Y_K	$L_K(6)$	$Y_K \cdot L_K(6)$
0	150	-0.20	-30.00
1	85	0.75	63.75
2	50	0.50	25.00
3	70	-0.05	-3.50
Σ	—	1.00	55.25

$P_3(6) = 221/4 = 55.25$ ms. La latencia media estimada al escalar el contenedor a 6 GB de RAM es de aproximadamente 55.25 milisegundos.

Verificación por la forma estándar: $-0.223958(216) + 7.093750(36) - 68.791667(6) + 261 = -48.375 + 255.375 - 412.75 + 261 = 55.25$ ms. Ambos caminos coinciden.

5. Parte B — Aplicativo de software

El aplicativo automatiza el proceso completo. Cumple los requerimientos funcionales de la guía: permite el ingreso dinámico de los nodos (x_i , y_i) y del punto a evaluar, implementa la lógica de Lagrange con bucles anidados de orden $O(n^2)$, muestra el valor interpolado y grafica los puntos experimentales junto con la curva continua de $P_3(x)$ en el intervalo $[2, 12]$, resaltando el punto $(6, P_3(6))$.

Motor de cálculo

```

"""
Análisis Numérico · Unidad 2 · Sesión 8
Interpolación de Lagrange – latencia media de un microservicio según la RAM asignada.

Datos de las pruebas de carga (stress testing):
  x (GB) :   2   4   8  12
  y (ms)  : 150  85  50  70

Objetivo: estimar la latencia cuando el contenedor se escala a x = 6 GB.

Fernando Istaña Flores · Universidad Peruana Unión
"""

from fractions import Fraction

# -----
# Motor de cálculo: polinomios base de Lagrange con bucles anidados  $O(n^2)$ 

```

```

# -----
def base_lagrange(xs, k, x):
    """
     $L_k(x) = \prod_{i \neq k} (x - x_i) / (x_k - x_i)$ 
    Un solo bucle por base; llamado para cada k queda  $O(n^2)$ .
    """
    Lk = 1.0
    for i in range(len(xs)):
        if i != k:
            Lk *= (x - xs[i]) / (xs[k] - xs[i])
    return Lk

def interpolar(xs, ys, x):
    """ $P_n(x) = \sum_k y_k * L_k(x)$ . Devuelve el valor y la lista de  $L_k(x)$ ."""
    bases = [base_lagrange(xs, k, x) for k in range(len(xs))]
    return sum(ys[k] * bases[k] for k in range(len(xs))), bases

# -----
# Coeficientes exactos de  $P_n$  en forma estándar (aritmética racional)
# -----
def coeficientes(xs, ys):
    """Expande la suma de Lagrange y devuelve  $[a_n, \dots, a_1, a_0]$  como fracciones."""
    n = len(xs)
    total = [Fraction(0)] * n
    for k in range(n):
        num = [Fraction(1)] # polinomio  $(x - x_i)$  acumulado
        den = Fraction(1)
        for i in range(n):
            if i == k:
                continue
            nuevo = [Fraction(0)] * (len(num) + 1)
            for j, c in enumerate(num):
                nuevo[j] += c # término  $x * num$ 
                nuevo[j + 1] -= c * xs[i] # término  $-x_i * num$ 
            num = nuevo
            den *= Fraction(xs[k] - xs[i])
        for j, c in enumerate(num):
            total[j] += Fraction(ys[k]) * c / den
    return total

def polinomio_texto(coef):
    grados = len(coef) - 1
    partes = []
    for j, c in enumerate(coef):
        g = grados - j
        if c == 0:
            continue
        signo = " + " if c > 0 and partes else " - " if c < 0 and partes else "-" if c < 0
        else ""
        val = abs(c)
        term = "%s" % val if g == 0 else ("%s·x" % val if g == 1 else "%s·x^%d" % (val, g))
        partes.append(signo + term)
    return "".join(partes)

if __name__ == "__main__":
    xs = [2, 4, 8, 12]
    ys = [150, 85, 50, 70]
    x_eval = 6

```

```

print("=" * 74)
print("DATOS EXPERIMENTALES")
print("=" * 74)
print("    k      x_k (GB)      y_k (ms)")
for k, (a, b) in enumerate(zip(xs, ys)):
    print(" %2d   %8.2f   %10.2f" % (k, a, b))

print()
print("=" * 74)
print("POLINOMIOS BASE EVALUADOS EN x = %g" % x_eval)
print("=" * 74)
p, bases = interpolar(xs, ys, x_eval)
for k, Lk in enumerate(bases):
    print("   L%d(%g) = %+10.6f      y%d · L%d = %+12.6f" % (k, x_eval, Lk, k, k, ys[k] * Lk))
print(" suma de las bases = %.6f   (debe ser 1)" % sum(bases))

print()
print("=" * 74)
print("POLINOMIO INTERPOLADOR EN FORMA ESTÁNDAR")
print("=" * 74)
coef = coeficientes(xs, ys)
print("   P3(x) = " + polinomio_texto(coef))
print(" coeficientes decimales: " + ", ".join("%+.6f" % float(c) for c in coef))

print()
print("=" * 74)
print("RESULTADO")
print("=" * 74)
print("   P3(%g) = %.4f ms" % (x_eval, p))
print()
print(" Verificación en los nodos:")
for a, b in zip(xs, ys):
    v, _ = interpolar(xs, ys, a)
    print("   P3(%2g) = %8.4f   (dato: %g)   error = %.2e" % (a, v, b, abs(v - b)))

# -----
# Gráfica: 4 puntos experimentales, curva de P3 en [2, 12] y el punto x=6
# -----
try:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    malla = [2 + i * (12 - 2) / 400 for i in range(401)]
    curva = [interpolar(xs, ys, t)[0] for t in malla]

    fig, ax = plt.subplots(figsize=(7.4, 4.3), dpi=200)
    ax.plot(malla, curva, color="#14366b", linewidth=2, label="P3(x) interpolante",
zorder=2)
    ax.scatter(xs, ys, s=48, color="#101c33", zorder=3, label="puntos medidos")
    ax.scatter([x_eval], [p], s=90, color="#c8890a", zorder=4,
                edgecolor="white", linewidth=1.4, label="P3(6) = %.2f ms" % p)
    ax.vlines(x_eval, min(curva) - 6, p, color="#c8890a", linestyle=":", linewidth=1.2,
zorder=1)
    ax.annotate("6 GB → %.2f ms" % p, (x_eval, p), textcoords="offset points",
                xytext=(10, 10), color="#8a6a12", fontsize=9)
    for a, b in zip(xs, ys):
        ax.annotate("(%g, %g)" % (a, b), (a, b), textcoords="offset points",
                    xytext=(6, -14), color="#3c4a63", fontsize=8)
    ax.set_xlabel("Memoria asignada x (GB)")
    ax.set_ylabel("Latencia media y (ms)")
    ax.set_title("Interpolación de Lagrange de la latencia frente a la RAM")
    ax.grid(color="#dfe5ef", linewidth=0.8)

```

```

ax.set_axisbelow(True)
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
ax.legend(frameon=False, fontsize=9)
fig.tight_layout()
fig.savefig("interpolacion_s8.png")
print()
print(" Gráfica guardada en interpolacion_s8.png")
except ImportError:
    print()
    print(" (matplotlib no está instalado: se omitió la gráfica)")

```

Salida del programa

DATOS EXPERIMENTALES

k	x_k (GB)	y_k (ms)
0	2.00	150.00
1	4.00	85.00
2	8.00	50.00
3	12.00	70.00

POLINOMIOS BASE EVALUADOS EN $x = 6$

```

L0(6) = -0.200000    y0 · L0 = -30.000000
L1(6) = +0.750000    y1 · L1 = +63.750000
L2(6) = +0.500000    y2 · L2 = +25.000000
L3(6) = -0.050000    y3 · L3 = -3.500000
suma de las bases = 1.000000 (debe ser 1)

```

POLINOMIO INTERPOLADOR EN FORMA ESTÁNDAR

```

P3(x) = -43/192·x^3 + 227/32·x^2 - 1651/24·x + 261
coeficientes decimales: -0.223958, +7.093750, -68.791667, +261.000000

```

RESULTADO

P3(6) = 55.2500 ms

Verificación en los nodos:

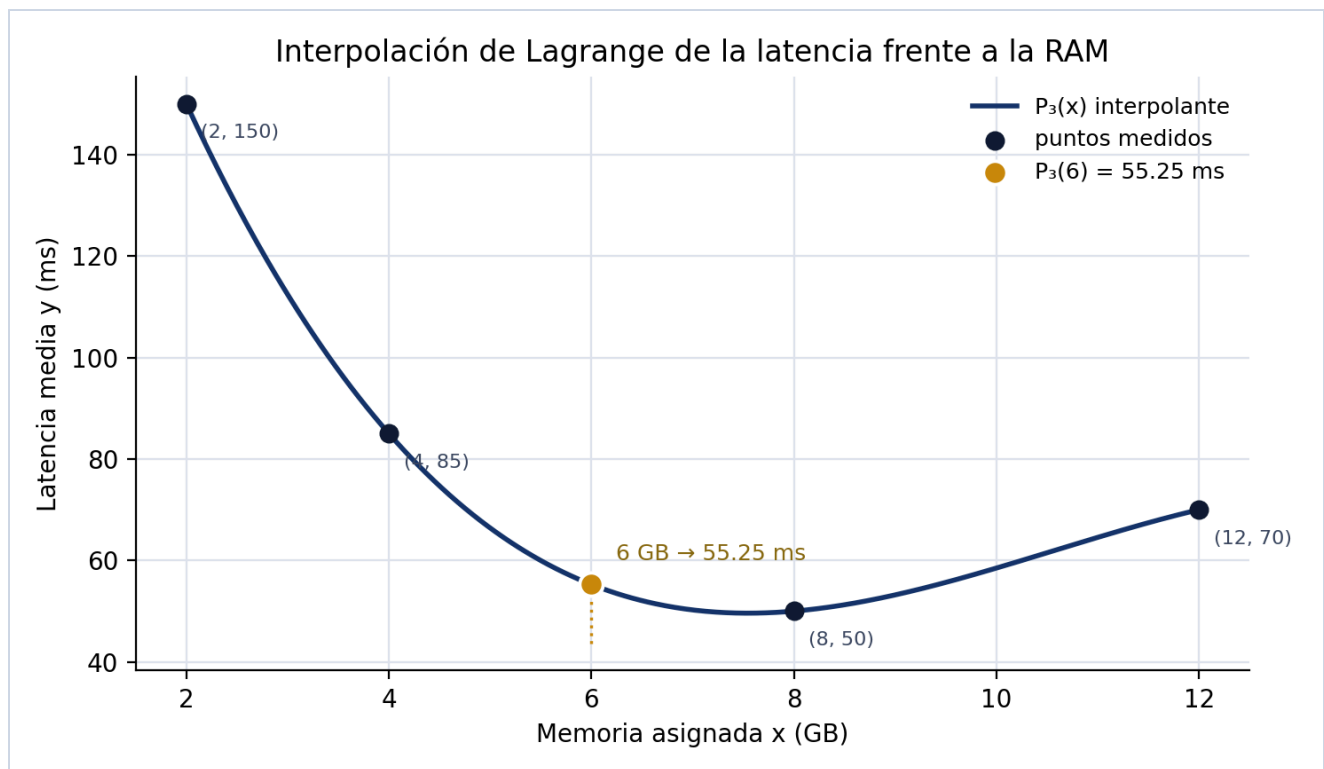
```

P3( 2) = 150.0000 (dato: 150) error = 0.00e+00
P3( 4) =  85.0000 (dato: 85)  error = 0.00e+00
P3( 8) =  50.0000 (dato: 50)  error = 0.00e+00
P3(12) =  70.0000 (dato: 70)  error = 0.00e+00

```

Gráfica guardada en interpolacion_s8.png

Interfaz gráfica



Puntos experimentales, curva de $P_3(x)$ sobre $[2, 12]$ y el punto interpolado $(6; 55.25)$ resaltado en ámbar.

6. Análisis de resultados

Lectura del resultado. Con 6 GB la latencia estimada baja a 55.25 ms frente a los 85 ms medidos con 4 GB, es decir, una mejora cercana al 35 %. Sin embargo, pasar de 6 a 8 GB solo aporta unos 5 ms adicionales (50 ms medidos), de modo que el retorno por gigabyte extra se agota rápidamente en esa zona.

Comportamiento del modelo. El polinomio reproduce la forma descrita en el enunciado: desciende con fuerza en el tramo inicial, alcanza un mínimo cerca de los 7.5 GB y vuelve a subir a partir de ahí, lo que es coherente con el efecto de la recolección de basura sobre heaps grandes. El punto $x = 6$ cae dentro del rango medido $[2, 12]$, por lo que se trata de una interpolación propiamente dicha y no de una extrapolación, caso en el que el error del polinomio crecería sin control.

Limitaciones. La interpolación polinómica pasa exactamente por los cuatro puntos, pero no garantiza el comportamiento entre ellos: con nodos muy separados un polinomio de grado alto puede oscilar (fenómeno de Runge). Con solo cuatro mediciones, la estimación debería contrastarse con una prueba de carga real a 6 GB antes de fijar la configuración en producción.

7. Conclusiones

Se construyeron las cuatro bases de Lagrange para los nodos medidos y se obtuvo el polinomio interpolador $P_3(x) = -(43/192)x^3 + (227/32)x^2 - (1651/24)x + 261$, que reproduce exactamente los valores experimentales. Su evaluación en $x = 6$ arroja una latencia media estimada de **55.25 ms**, resultado confirmado tanto por el reemplazo numérico manual como por el aplicativo desarrollado. La curva muestra que el beneficio de

asignar más memoria se concentra entre los 2 y los 6 GB y que, más allá del mínimo cercano a 7.5 GB, ampliar el contenedor empeora la latencia en lugar de mejorarla.